Programarea Calculatoarelor și Limbaje de Programare 1

Tema - Password Manager

Data postării: 22.11.2025

Ultima actualizare: 11.12.2025, ora 10:00

Deadline: 19.12.2025 ora 23:55

Echipă temă:
Carmen ODUBĂȘTEANU, Mihai NAN
George-Alexandru TUDOR



Facultatea de Automatică şi Calculatoare

Universitatea Națională de Știință și Tehnologie Politehnica București

Anul universitar 2025 - 2026

Seria CC

1 Introducere - Managerul de parole

În februarie 2022, **National Security Agency (NSA)** a publicat raportul tehnic *Cisco Password Types: Best Practices*, în care analiza metodele de protejare a parolelor utilizate în dispozitivele de rețea. **Documentul** pornea de la un incident semnalat de Department of Homeland Security (DHS), unde atacatori cibernetici au reușit să extragă valori hash ale parolelor și alte informații sensibile din fișierele de configurare ale infrastructurii.

După obținerea acestor valori, adversarii au reușit să compromită rețele întregi, arătând consecințele directe ale utilizării de algoritmi slabi și ale reutilizării parolelor.

Pe scurt, raportul arată un adevăr valabil dincolo de mediul guvernamental: o parolă slabă sau reutilizată nu este doar o eroare individuală, ci o vulnerabilitate structurală. Odată ce un atacator obține o parolă (sau hash-ul acesteia), acesta o poate folosi pentru a accesa și alte sisteme care reutilizează acea parolă.

Aceleași principii se aplică și în sfera utilizatorilor obișnuiți. Într-un context în care fiecare aplicație sau site cere o autentificare, majoritatea oamenilor aleg soluția comodă: o parolă ușor de ținut minte, dar ușor de ghicit.

Una dintre soluțiile existente în acest sens este conceptul de **password manager** care:

- generează parole unice și puternice, greu de reținut;
- le stochează într-o formă criptată, folosind o cheie de protecție (master password);
- le oferă automat la autentificare, eliminând nevoia de a le memora.

Managerul de parole nu elimină vulnerabilitățile umane, dar le izolează: compromiterea unui cont nu mai duce automat la compromiterea întregului sistem.

Această temă simulează, într-o manieră simplistă, o parte din principiile de funcționare ale unui manager de parole, printr-o serie de exerciții implementate în **limbajul C**.

2 Cerința I - Criptarea parolelor: Cifrul Vigenère

Protejarea datelor sensibile reprezintă o parte esențială a oricărui sistem de securitate.

O regulă fundamentală în domeniu este că **informațiile confidențiale nu trebuie niciodată stocate în clar** - mai ales parolele. În practică, acest principiu se realizează prin aplicarea unor **algoritmi de criptare** care transformă textul lizibil într-o formă imposibil de interpretat fără cheia corectă.

Una dintre cele mai vechi și mai elegante metode de criptare este **cifrul lui Vigenère** în care fiecare literă a textului este deplasată în alfabet cu un număr de poziții determinat de o cheie secretă. Această abordare introduce o corelație între pozițiile cheii și textul clar, rezultând un mesaj criptat greu de decodat fără cunoașterea exactă a cheii.

În această cerință, vom lucra cu o **variantă modificată a cifrului Vigenère**, folosind toate literele mici și mari ale alfabetului englez (a–z, A–Z). Astfel, după litera z se continuă cu A, iar după Z ciclul revine la a. Această modificare extinde spațiul de criptare de la 26 la 52 de simboluri posibile.

Algoritmul trebuie să urmeze pașii de mai jos:

1. **Extinderea cheii.** Cheia de criptare se repetă (prin concatenare) până când va avea aceeași lungime ca textul. Dacă lungimea cheii este mai mare decât lungimea textului, se vor folosi doar caracterele corespunzătoare din cheie.
2. **Deplasarea pe poziții.** Pentru fiecare poziție $\boldsymbol{i}$:
   - se determină **deplasarea** dată de litera cheii de pe poziția $\boldsymbol{i}$, egală cu distanța ei față de litera $\boldsymbol{a}$ în alfabetul extins;
   - litera textului clar de pe poziția $\boldsymbol{i}$ este rotită spre dreapta cu această deplasare în același alfabet extins.

3. **Ordinea alfabetului (52 de simboluri).** Alfabetul este ordonat astfel: ***a, b, . . . , z, A, B, . . . , Z***. Dacă, în urma deplasării, se depășește ***z***, se continuă numărarea cu ***A***; dacă se depășește ***Z***, se revine la ***a*** (rotire circulară).

Având la dispoziție **o cheie** și **un text în clar**, calculați **textul criptat** folosind **varianta modificată a cifrului Vigenère** descrisă mai sus (alfabet de 52 de litere, cheie extinsă, rotire circulară).

Specificații

Date de intrare: un șir de caractere reprezentând **cheia**, urmat de un șir de caractere reprezentând **textul clar**. Fiecare șir se află pe câte o **linie separată** și vor fi prelucrate doar dacă sunt formate exclusiv din litere (a–z, A–Z).

Date de ieșire: un șir de caractere ce reprezintă textul criptat, dacă datele de intrare sunt valide, sau textul `INVALID` în caz contrar.

Indicație

Se va scrie o funcție care verifică dacă un șir conține doar litere. Dacă șirul conține alte caractere (cifre, spații, simboluri etc.), acesta este considerat **invalid**. Funcția va fi aplicată atât asupra **cheii**, cât și asupra **textului clar**.

Restricții

Dimensiunea maximă a fiecărui șir de caractere (cheie, text, rezultat) este de **14.000** caractere.

Exemplul 1

Intrare:

```
bcaA
AzYNZbccMn
```

Ieșire:

```
BBYnadcCNp
```

Explicație:
În acest exemplu, cheia de criptare este `bcaA`, iar textul clar este `AzYNZbccMn`. Deoarece lungimea cheii este mai mică decât lungimea textului (4 față de 10 caractere), cheia trebuie **extinsă prin repetare** până acoperă complet textul. Astfel, cheia folosită efectiv devine:

`bcaAbcaAbc`

Criptarea se realizează caracter cu caracter, folosind deplasările din alfabetul extins `[a-zA-Z]` (52 de simboluri).

1. Primul caracter: Cheie: `b` → se află la o distanță de `1` față de litera `a`. Text: `A`. Aplicăm deplasarea de 1 poziție în alfabetul extins, deci `A` → `B`.

2. Al doilea caracter: Cheie: `c` → se află la distanța 2 față de `a`. Text: `z`. Deplasăm 2 poziții: după `z` urmează `A`, apoi `B` - rezultatul este `B`.

3. Al treilea caracter: Cheie: `a` → deplasarea este 0. Text: `Y`. Deoarece deplasarea este nulă, caracterul rămâne neschimbat - `Y`.

4. Al patrulea caracter: Cheie: `A` → în alfabetul extins, aceasta se află la poziția 26 (imediat după `z`), deci deplasarea este 26. Text: `N`. Se deplasează 26 poziții spre dreapta; în acest caz, depășim limitele majusculelor și revenim la litere mici. Rezultatul este `n`.

Procedând astfel pentru fiecare pereche de caractere, se obține următoarea corespondență completă:

Text clar:	A	z	Y	N	Z	b	c	c	M	n
Cheie:	b	c	a	A	b	c	a	A	b	c
Rezultat:	B	B	Y	n	a	d	c	C	N	p

Prin concatenarea caracterelor rezultate se obține textul criptat final:

`BBYnadcCNp`

Observați cum, datorită structurii alfabetului extins (unde după `z` urmează `A`), deplasările trec liber din zona literelor mici în cea a literelor mari și invers, fără pierderi de informație. Astfel, algoritmul produce o criptare circulară uniformă, care tratează toate literele în mod consistent.

Exemplul 2 (cheie invalidă)

Intrare:

```
bca2
AzYNZbccMn
```

Ieșire:

```
INVALID
```

Explicație: Cheia conține caracterul `2`, care nu este o literă, deci intrarea este invalidă.

Exemplul 3 (text invalid)

Intrare:

```
bca
AzYNZbcc)n
```

Ieșire:

```
INVALID
```

Explicație: Textul în clar conține caracterul `)`, care nu face parte din alfabetul valid.

3 Cerința II – Completarea automată a datelor (AutoFill)

O altă funcționalitate importantă a unui **manager de parole** este capacitatea de a completa automat datele utilizatorului în diverse formulare online. De la simple câmpuri precum *nume*, *adresă de e-mail* sau *număr de telefon*, până la formulare complexe de autentificare sau plată, mecanismul de **autofill** simplifică experiența utilizatorului și reduce riscul de erori manuale.

În esență, această operație constă în identificarea anumitor **șabloane de câmpuri** (precum `NUME`, `EMAIL`, `ADRESA`) într-un text și înlocuirea lor cu valorile corespunzătoare stocate anterior de managerul de parole. Acest proces automatizat elimină nevoia de a introduce manual datele de fiecare dată și reproduce comportamentul real al aplicațiilor de gestionare a identității digitale.

Sarcina voastră este să implementați această operație de **AutoFill** pe un text (formular), primind la intrare o listă de câmpuri și valorile cu care acestea trebuie completate. Scopul este ca programul să recunoască aceste câmpuri în text și să le înlocuiască dinamic, obținând un rezultat final completat automat.

Specificații

Date de intrare:

- Un număr întreg **N**, reprezentând numărul de perechi `(câmp - valoare)`.
- Urmează **N linii**, fiecare conținând denumirea câmpului și valoarea asociată, sub forma a două șiruri de caractere separate printr-un spațiu.
- După acestea, urmează textul formularului asupra căruia se va aplica funcția de **AutoFill**.

Textul formularului poate avea o lungime maximă de **11.000 caractere**.

Date de ieșire: Textul formularului, în care fiecare câmp este înlocuit cu valoarea corespunzătoare din lista de perechi.

Restricții și cerințe de implementare

În implementarea voastră trebuie utilizată următoarea structură de date:

```
struct pair {
    char *field;
    char *value;
};
```

- Perechile vor fi memorate într-un **vector alocat dinamic**, având exact atâta memorie cât este necesar pentru cele **N** perechi citite.
- Fiecare șir de caractere din cadrul unei perechi (`field`, `value`) trebuie de asemenea alocat dinamic, folosind o dimensiune egală cu numărul de caractere efectiv necesar (fără alocări în exces).
- Textul rezultat în urma operației de **AutoFill** trebuie să respecte aceeași regulă: alocare exactă, nici mai multă, nici mai puțină memorie decât este necesar.

Exemplul 1

Intrare:

```
3
NUME Pop
PRENUME Ion
MAIL ion.pop@email.com
Numele meu este NUME PRENUME si am email MAIL.
```

Ieșire:

```
Numele meu este Pop Ion si am email ion.pop@email.com.
```

Explicație: Sunt citite 3 perechi (câmp → valoare), apoi se aplică **AutoFill** pe textul formularului. Toate aparițiile câmpurilor `NUME`, `PRENUME`, `MAIL` sunt înlocuite cu valorile corespunzătoare, rezultând textul final din secțiunea „Ieșire”.

Observații

Înlocuirile sunt *case-sensitive* și se aplică pentru *fiecare apariție* a câmpului în text.

Un câmp este înlocuit doar dacă apare ca *un cuvânt întreg*, adică delimitat de spații, început/sfârșit de text sau semne de punctuație. De exemplu, în textul `"Am un NUME. NUMEle este unicat."`, se va înlocui doar prima apariție.

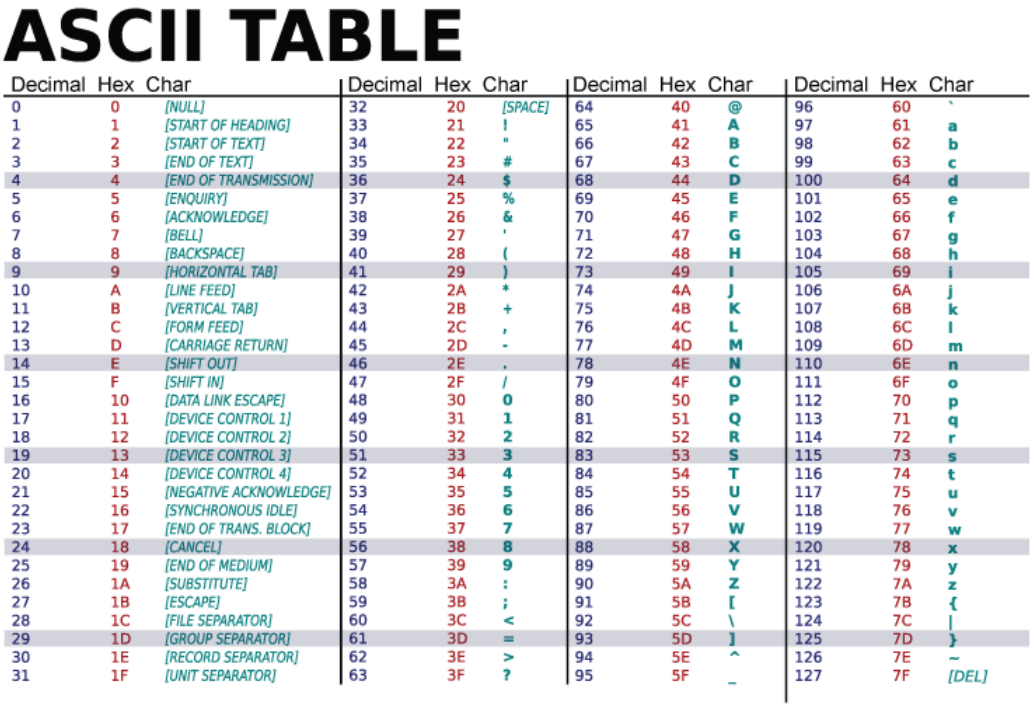
4 Cerința III – Generarea parolelor sigure

Securitatea oricărui sistem informatic depinde, în mare măsură, de **calitatea parolelor** utilizate pentru autentificare. O parolă eficientă trebuie să fie **imprevizibilă**, **complexă** și **unică**, astfel încât să nu poată fi ghicită prin încercări automate sau dedusă din informații personale.

Parolele slabe - precum nume proprii, date de naștere, combinații simple de caractere (`123456`, `password`) sau termeni asociați utilizatorului - pot fi sparte cu ușurință prin atacuri de tip *dictionary* sau *brute-force*. Pentru a evita astfel de vulnerabilități, sistemele moderne folosesc generatoare automate de parole, capabile să producă șiruri de caractere **puternice din punct de vedere computațional** și **independente de utilizator**.

Un **Password Manager** implementează exact acest principiu, oferind utilizatorului parole sigure, greu de reprodus sau intuit. În această cerință, veți simula procesul de **generare automată a parolelor** conform unui set de reguli prestabilite, pornind de la un **număr aleator (seed)** și o **lungime specificată** a parolei. Rezultatul final va fi o parolă unică, obținută prin aplicarea următoarelor transformări:

1. **Generarea șirului inițial de caractere.** Pornind de la un număr întreg `seed` primit ca dată de intrare, se va genera aleator (folosind funcția `rand()`) un șir de caractere de o dimensiune precizată. Șirul rezultat va conține exclusiv caracterele ale căror coduri ASCII sunt cuprinse între **32 și 125** (inclusiv).



ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Figura 1: Tabel ASCII

2. **Aplicarea unei funcții de codificare asupra fiecărui caracter.** Pentru fiecare caracter generat, se va alege **aleator** una dintre cele trei funcții de codificare de mai jos:

   (a) **Codificarea 1 - bazată pe numărul de apariții.** Fiecare caracter este înlocuit cu concatenarea dintre:
   - caracterul original
   - numărul de apariții ale acelui caracter în șir **până la poziția curentă** (caracterul curent nu se numără).

   Exemplu:

   `nnan → n0n1a0n2`

   (b) **Codificarea 2 - transformări pe biți.** Asupra codului ASCII al caracterului (**pe 8 biți**) se aplică următoarele operații:

   i. se neagă bitii de pe pozițiile 3 și 6 (numărate de la LSB – Least Significant Bit);

   ii. se calculează numărul total de biți egali cu 1 din rezultatul obținut;

   iii. se aplică operația **OR logic (SAU)** cu masca `00100000`;

   iv. se setează la 0 bitul de pe poziția 0 (LSB).

   Caracterul final va fi format prin concatenarea:
   - caracterului obținut după aceste transformări;
   - numărului de biți de 1 determinați anterior operației OR.

Exemplu detaliat:

```
Caracter: c (cod ASCII 01100011)
1. 01100011 → 00101011
2. Numărul de biți de 1 = 4
3. 00101011 OR 00100000 → 00101011
4. Setăm LSB la 0 → 00101010 ('*')
Rezultat final: *4
```

(c) **Codificarea 3 - conversie de litere.** Literele mici se transformă în litere mari. Alte caractere (cifre, simboluri, litere deja mari) rămân nemodificate.

Specificații

Date de intrare:
- Două numere întregi: **seed** (valoarea de pornire pentru generarea aleatoare) și **L** (dimensiunea șirului de caractere generat).

Date de ieșire:
- Parola generată, obținută după aplicarea transformărilor specificate.

Restricții și cerințe de implementare

- Generarea parolei va fi **separată în mai multe funcții**:
  - o funcție care **generează șirul aleator**;
  - o funcție care **transformă șirul generat**;
  - funcții separate pentru **fiecare tip de codificare** (1, 2 și 3).
- Toate codificările posibile asupra unui caracter vor fi reținute într-un **vector de pointeri la funcții**, vector care va fi transmis ca argument funcției de **codificare a șirului** generat aleator.
- Alegerea codificării aplicate caracterului curent se va face **aleator**: se generează un index din mulțimea `{0, 1, 2}` corespunzător uneia dintre funcțiile din vector (**Obs.** indexul `0` corespunde codificării **1**, etc.).
- **F**iecare funcție primește ca argument un **caracter** și întoarce un **pointer către un șir de caractere** (rezultatul codificării pentru caracterul respectiv).
- Pentru calculul **numărului de apariții** ale caracterelor (folosit în codificarea 1) se va utiliza un **vector alocat static** în interiorul funcției care realizează codificarea 1. În acest vector se va completa, pentru fiecare caracter posibil (cod ASCII în [**32..125**]), numărul de apariții la un moment dat.
- **Gestionarea memoriei**:
  - alocați **exact** memoria necesară pentru șirurile rezultate (fără surplus);
  - **eliberați** memoria oricărui șir alocat dinamic imediat ce nu mai este utilizat.

Exemplul 1

Intrare:

```
10 4
```

Ieșire:

```
)0\F*3
```

Explicație: Pornind de la **seedul 10** și dimensiunea **4**, programul generează aleator un șir de caractere de lungime 4. Șirul generat este:

```
)\FC
```

Pentru fiecare caracter se alege aleator una dintre cele trei funcții de codificare:

1. Caracterul `)`. Se aplică **codificarea 1**. Deoarece caracterul `)` nu a mai apărut anterior în șir, numărul de apariții este 0. Rezultatul este concatenarea caracterului cu acest număr: `)  → )0`

2. Caracterul \ . Se aplică **codificarea 3**. Cum caracterul \ nu este o literă, nu se produce nicio modificare. Rezultatul rămâne același: \.

3. Caracterul F. Se aplică din nou **codificarea 3**. Deci, caracterul **F** rămâne nemodificat.

4. Caracterul C. Se aplică **codificarea 2**.

```
Caracter: C (cod ASCII 01000011)
1. 01000011 → 00001011
2. Numărul de biți de 1 = 3
3. 00001011 OR 00100000 → 00101011
4. Setăm LSB la 0 → 00101010 ('*')
Rezultat final: *3
```

Concatenând rezultatele individuale se obține parola finală:

```
)0\F*3
```

Astfel, fiecare caracter a fost procesat independent printr-o codificare aleatorie, rezultând o parolă unică și greu de reprodus.

5 Rulare, testare și predare

Reguli de rulare și testare

- Toate **intrările** se citesc *de la tastatură* (stdin). **Nu** folosiți fișiere în cod.
- Pentru a evita introducerea manuală a datelor la fiecare rulare, puteți redirecționa intrarea dintr-un fișier:
  `./nume_executabil < nume_fisier_intrare`
- Fiecare problemă se scrie într-un **fișier sursă separat** numit `problemaX.c`, unde `X` ∈ `{1,2,3}`.
- Includeți un fișier `README.md` cu:
  - timpul estimat de implementare;
  - **pe scurt** descrierea soluției pe care ați ales-o pentru fiecare problemă și alte lucruri pe care le considerați utile de menționat (codul trebuie să fie documentat suficient prin comentarii).
  - păstrați **linii ≤ 80 de caractere**.
- **Pentru notare trebuie să încărcați o arhivă:** `GRUPANumePrenume.zip` (ex.: `311CCPopIon.zip`) conținând **direct în rădăcină**: `problema1.c`, `problema2.c`, `problema3.c`, `README.md`.
- **Temele vor fi încărcate** în slotul special creat pe Moodle.

Observații

Sfat practic: folosiți `-Wall` la compilare pentru a prinde avertizările timpuriu. Puteți crea fișiere de test diferite și rula cu redirecționare: `./problema1 < test1.in`.

Observații

Toate rezultatele trebuie să conțină o linie goală la final. După ce afișați rezultatul, afișați, pe o linie separată, încă un "\n".

Checker oficial și mod de utilizare

Pentru verificarea automată a soluțiilor este pus la dispoziție un script de testare denumit `checker.sh`. Acesta compilează toate sursele folosind `make` și rulează testele aferente fiecărei cerințe. Pentru a rula checker-ul trebuie să oferiți permisiuni fișierului check.sh (de exemplu, chmod +x check.sh).

Structura testelor:
- `tests/problemaX/in/` - fișiere de intrare (`testYY.in`);
- `tests/problemaX/ref/` - fișiere de ieșire de referință (`testYY.ref`);
- `tests/problemaX/out/` - fișierele rezultate în urma rulării propriului cod (`testYY.out`).

Mod de rulare:
- Rulare completă (toate cerințele):
  `./checker.sh`
- Rulare pentru o singură cerință (ex. doar problema 2):
  `./checker.sh 2`

Scriptul verifică automat:
- corectitudinea rezultatelor (folosind `diff`);
- scurgerile de memorie (prin `valgrind`);
- respectarea timpului-limită de execuție (prin `timeout`).

Rezultatele testelor sunt afișate individual pe ecran, împreună cu punctajul obținut pentru fiecare cerință și totalul cumulat.

6 Restricții, structură și coding style

Restricții de implementare

- **Modularizare obligatorie:** separați logica în **mai multe funcții**, conform enunțurilor. Sursele **nu vor fi punctate** dacă tot programul este scris în `main()`.
- **Este strict interzisă folosirea variabilelor globale.**
- Soluția se scrie în **limbajul C**. **Nu** folosiți sintaxă/instrucțiuni specifice **C++**.
- **Afișarea rezultatului se va face în funcția main, NU în funcțiile auxiliare. Funcțiile auxiliare doar returnează rezultate, nu afișează nimic pe ecran.**

Coding style și bune practici

- Alegeți **denumiri semnificative** pentru variabile și funcții.
- Evitați linii goale în exces, spații la final de linie (*trailing spaces*) și aliniere inconsistentă.
- Mențineți lungimea liniilor la $\leq$ **80** de caractere (atât în cod, cât și în `README`).
- Păstrați același stil peste tot (indentare, acolade, spațieri).
- Comentați porțiunile de cod ce conțin o logică mai greu de urmărit; nu repetați în comentarii ce este evident din cod și plasați comentariile deasupra blocului pe care îl descrieți.
- Puteți folosi formatare automată, dar ideal este să respectați stilul *în timp ce* scrieți codul.

7 Reguli de integritate și evaluare

Reguli generale

- Temele sunt **strict individuale**. Orice formă de copiere, partajare de cod, colaborare directă între studenți sau inspirația din surse externe este **strict interzisă**.
- Persoanele care prezintă **porțiuni identice de cod** - indiferent de sursă sau intenție - vor primi **0 puncte** pe întreaga temă.
- Nu copiați cod de pe Internet! Se pot detecta automat asemănări între soluții diferite, inclusiv atunci când codul provine din surse publice (GitHub, forumuri, AI etc.).
- Temele trimise **după deadline** nu vor fi evaluate.
- Pentru întrebări despre temă, folosiți **exclusiv forumul oficial al temei**. Verificați periodic forumul - pot apărea clarificări suplimentare utile.

Verificări tehnice și bune practici

- Compilarea nu trebuie să producă avertizări (**warnings**). Testați codul cu flagul: `gcc -Wall problemaX.c -o problemaX`
- Eliberați toată memoria alocată dinamic. Pentru verificare, rulați: `valgrind --tool=memcheck --leak-check=full ./problemaX`
- Compilați programul cu opțiunea `-g` pentru a putea urmări liniile care generează erori sau scurgeri de memorie.

Atenție!

Nerespectarea regulilor din această secțiune **(și din toate subsecțiunile cu restricții)** va duce automat la **punctaj 0 pe întreaga temă**. Nu se vor face excepții.

8 Notare

Cerința	Puncte
Cerința I - Criptarea parolelor: Cifrul Vigenère	35
Cerința II - Completarea automată a datelor (AutoFill)	40
Cerința III - Generarea parolelor sigure	45
Total - 1.2p din nota finală	**120**

Atenție!

Punctajul afișat de **checker** reprezintă punctajul de bază de la care se pornește corectarea. **Corectorii își rezervă dreptul de a scădea puncte pentru orice problemă găsită în implementare, dacă vor considera acest lucru necesar.** De exemplu:

- sunt încălcate **restricțiile de implementare** (ex.: variabile globale, lipsa modularizării etc.);
- nu sunt respectate **cerințele de alocare/eliberare corectă a memoriei**;
- codul nu respectă **stilul de programare și bunele practici** menționate;
- soluția prezintă diferențe față de specificațiile din enunț (comportament, formatul ieșirii, etc.).

Cu alte cuvinte, **nota de la care se pornește este cea de pe checker**, dar aceasta poate fi modificată în urma corectării manuale.

Depunctările care se pot aplica:

- 5 puncte pentru lipsa README sau explicații superficiale;
- 10 puncte pentru Coding Style necorespunzător (include și lipsa comentariilor în cod - nu trebuie comentată fiecare linie, doar ceea ce este esențial pentru a putea fi ușor de înțeles rezolvarea);
- Depunctare totală pentru formatarea incorectă a arhivei (alt nume, alt tip, alte nume pentru fișiere, alte fișiere în plus sau în minus) - **0 puncte pe temă**;
- **O temă care NU compilează va fi punctată cu 0.**